

CS201 Midterm 3

6 August 2025

Olcay Taner YILDIZ

Abstract— Write only ONE function per question. Do not check the input, assume that the input is correct. Unless noted,

- (i) The input data structure is not empty
- (ii) Time complexity of the algorithm does not matter
- (iii) You can not use extra data structures including arrays
- (iv) You can use any function given in the data structure
- (v) You can not modify the data structure contents

I. QUESTION (HEAP)

Write a recursive method in **MinDHeap** class

```
void ascendants(int current,
                int* list,
                int& index)
```

which fills the ascendants (including also current) of the HeapNode with index *current* to the array list. Use and modify index to store the HeapNodes into correct positions. Your method should run in $\mathcal{O}(\log N)$ time. You are not allowed to use any class method.

II. QUESTION (HEAP)

Write the method in MaxHeap class

```
int third()
```

that returns the third maximum number in the heap. Your method should run in $\mathcal{O}(1)$ time. You are not allowed to use any class method except `getData`.

III. QUESTION (DISJOINT SET)

Given in the index of a set as *current*, write a recursive method

```
void ascendants(int current,
                int* list,
                int& index)
```

that fills the ascendants (including also current) of that set to the array list. Use and modify index to store the indexes of the sets into correct positions. You are not allowed to use any class method except `getParent`.

IV. QUESTION (DISJOINT SET)

Given the index of a set, write a recursive function that finds the value of that set. The value of a node is $1 + \text{maximum value of its children}$. Any node with no children has value 1. You are not allowed to use any class method except `getParent`.

```
int value(int index)
```

V. QUESTION (GRAPH)

Given the adjacency list representation of a graph *G*, write a method which returns true if there are two nodes whose outgoing node lists are the same, false otherwise. Assume that the outgoing node lists are sorted. Your method should run in $\mathcal{O}(V^3)$ time.

```
bool outgoingListSame()
```

VI. QUESTION (GRAPH)

The adjacency matrix *M* represents the number of ways to travel between pairs of nodes in a graph in exactly one move. M^k represents the number of ways to travel between pair of nodes in a graph in exactly *k* moves. Write the method

```
int** numberOfWaysInTwoMoves()
```

which calculates and returns M^2 for a graph. Your method should run in $\mathcal{O}(V^3)$ time.